Projekt przejściowy

Temat projektu

Wykorzystanie metod uczenia maszynowego do wykrywania anomalii w danych pochodzących z sieci czujników IoT oraz opracowanie systemu monitorowania, wizualizacji i raportowania wykrytych nieprawidłowości.

Skład grupy

1. Mateusz Buchczyk 272816
2. Remigiusz Szmyt 272819

1 Przegląd literatury i rozwiązań technicznych

Przegląd obejmuje cztery książki podręcznikowe, cztery publikacje naukowe i pięć źródeł technicznych dotyczących architektury ESP32–MQTT–serwer–HMI oraz detekcji anomalii.

1.1 Podstawy teoretyczne i podręczniki

Wybrane podręczniki porządkują podstawy uczenia maszynowego oraz metodykę pracy z danymi. Stanowią punkt wyjścia do omówienia algorytmów detekcji i uzasadnienia sposobu ich porównania w projekcie.

1.1.1 Russell i Norvig: Artificial Intelligence: A Modern Approach

Podręcznik Russella i Norviga przedstawia sztuczną inteligencję jako dziedzinę obejmującą m.in. rozumowanie, podejmowanie decyzji, reprezentację wiedzy i uczenie. Dla projektu istotne są części dotyczące uczenia na przykładach, modeli probabilistycznych oraz niepewności [1].

Źródło wybrano jako podstawę pojęciową projektu. Pomaga wyjaśnić, dlaczego system uczący się na pomiarach różni się od zestawu ręcznie ustalonych progów temperatury lub oświetlenia. Jego rola jest ogólna: szczegółowy opis Isolation Forest i One-Class SVM oparto na oryginalnych publikacjach, a nie na samym podręczniku. Taki podział pozwala zachować właściwy zakres cytowań i nie przypisywać książce szczegółów konkretnej implementacji.

W projekcie wiedza ta pomaga oddzielić trzy poziomy: odczyt z ESP32, ocenę nietypowości wektora pomiarowego oraz reakcję interfejsu HMI. Wniosek projektowy jest następujący: alarm modelu należy prezentować jako sygnał wymagający interpretacji, a nie automatyczne rozpoznanie uszkodzenia czujnika. Zmiana warunków otoczenia może być rzeczywista i poprawnie zmierzona, mimo że model uzna ją za nietypową. Podręcznik daje zatem ramy do opisania celu systemu, zakresu jego autonomii i ograniczeń wnioskowania na podstawie danych.

1.1.2 Zhou: Machine Learning

Książka Zhi-Hua Zhou obejmuje podstawy uczenia maszynowego, wybór i ocenę modeli, drzewa decyzyjne, maszyny wektorów nośnych, uczenie zespołowe, grupowanie oraz dobór cech. Oficjalny spis treści wskazuje osobne rozdziały poświęcone SVM, ocenie modeli i selekcji cech. Są to obszary bezpośrednio związane z planowanym porównaniem detektorów [2].

Pozycję wybrano, ponieważ zapewnia wspólny język opisu dwóch odmiennych podejść: zespołu losowych drzew oraz metody wykorzystującej jądro. Pozwala omówić reprezentację próbek, generalizację i dobór parametrów przed przejściem do specyfiki detekcji anomalii. Rozdział o klasycznym SVM nie zastępuje jednak wyprowadzenia wariantu jednoklasowego, ponieważ ten wymaga odrębnego odwołania do pracy Schölkopfa i współautorów.

Konkretnym wkładem w projekt jest uporządkowanie planu porównania modeli na tych samych sześciu cechach pomiarowych. Temperatura, wilgotność, ciśnienie, oświetlenie i dwie miary przyspieszenia mają odmienne skale oraz znaczenie fizyczne. Wiedza o cechach i modelach pomaga uzasadnić analizę skalowania przed One-Class SVM oraz

ocenę przydatności każdego kanału. Dobór parametrów powinien być oddzielony od końcowego testu. Są to decyzje metodyczne projektu, nie gotowe zalecenia książki dotyczące konkretnie ESP32 ani potwierdzenie optymalności przyjętego podziału 70/15/15.

1.1.3 Goodfellow, Bengio i Courville: Deep Learning

Podręcznik omawia matematyczne podstawy uczenia maszynowego, sieci głębokie, regularyzację, optymalizację i uczenie reprezentacji. Dla analizowanego systemu szczególnie użyteczne są rozdziały o modelowaniu sekwencji, metodyce praktycznej i autoenkoderach. Autoenkoder przekształca dane do reprezentacji wewnętrznej i odtwarza wejście, a ograniczenia architektury lub procesu uczenia mają prowadzić do uchwycenia użytecznych właściwości danych [3].

Książkę wybrano jako teoretyczne tło alternatywy dla Isolation Forest i One-Class SVM. Umożliwia wyjaśnienie, czym różni się ręcznie przygotowany wektor pomiarów od reprezentacji uczonej przez sieć. Nie stanowi jednak samodzielnego dowodu, że duży błąd rekonstrukcji zawsze oznacza awarię, ani że metoda głęboka będzie skuteczniejsza od prostszych detektorów. Zastosowanie rekonstrukcji do anomalii czasowych rozwija osobno omówiona publikacja Malhotry i współautorów.

Wkład w projekt polega przede wszystkim na świadomym określeniu jego zakresu. Przy około 39 godzinach zarejestrowanych pomiarów liczba wektorów nie jest równoznaczna z dużą różnorodnością stanów pracy. Wdrożenie autoenkodera wymagałoby dodatkowego uzasadnienia architektury, regularyzacji i procedury uczenia. Dlatego książka wspiera opis możliwego rozszerzenia systemu, bez deklarowania wykonanych eksperymentów z sieciami. Dla HMI wynika stąd potrzeba prezentowania rodzaju modelu i znaczenia jego wyniku: błąd rekonstrukcji, wynik izolacji i funkcja decyzyjna SVM nie są tą samą wielkością i nie powinny być bez uzasadnienia porównywane na wspólnej skali.

1.1.4 Géron: Hands-On Machine Learning

Trzecie wydanie książki Gérona przedstawia praktyczny przebieg projektu uczenia maszynowego z użyciem scikit-learn, Keras i TensorFlow. Towarzyszące mu materiały autora obejmują przygotowanie danych, klasyfikację, SVM, drzewa, metody zespołowe i uczenie bez nadzoru. Zaletą publikacji jest powiązanie pojęć z wykonywalnymi przykładami w notatnikach Python [4].

Pozycję wybrano ze względu na przełożenie metodyki ML na organizację kodu i eksperymentów. W projekcie potrzebny jest powtarzalny ciąg przekształceń od tabeli pomiarów do predykcji, a nie jednorazowe uruchomienie algorytmu. Przykłady nie dotyczą dokładnie badanego zestawu czujników, dlatego powinny być traktowane jako wzorce pracy, a nie gotowe rozwiązanie problemu. W szczególności sposób podziału danych trzeba dostosować do ich chronologicznego charakteru.

Konkretnym zastosowaniem jest przygotowanie wspólnego procesu skalowania i inferencji dla One-Class SVM: parametry przekształceń należy wyznaczać na treningu, a potem stosować bez ponownego dopasowania do walidacji, testu i nowych wiadomości z serwera MQTT. Warto utrwalić model razem z kolejnością cech i konfiguracją przetwarzania, aby HMI pokazywał wyniki zgodne z eksperymentem. Książka pomaga również uzasadnić rozdzielenie strojenia od oceny końcowej. Jej wykorzystanie nie oznacza przyjęcia wszystkich bibliotek z tytułu: TensorFlow i Keras nie są konieczne do porównania Isolation Forest z One-Class SVM.

1.2 Publikacje naukowe

Poniższe publikacje przedstawiają podstawy badanych detektorów, przykład ich zastosowania w IoT oraz metody analizy szeregów czasowych. Ich omówienie łączy opis metod z oceną przydatności dla projektowanego systemu.

1.2.1 Liu, Ting i Zhou: Isolation Forest

Autorzy proponują detekcję opartą na izolowaniu obserwacji przez losowe podziały przestrzeni cech. W drzewie wybierana jest cecha i punkt podziału, a obserwacje rzadkie i odmienne od większości zwykle można odizolować mniejszą liczbą podziałów. Wynik lasu wynika ze średniej długości ścieżki, z odpowiednią normalizacją. Publikacja analizuje również próbkowanie danych oraz koszt obliczeń i porównuje metodę z innymi detektorami, wykorzystując m.in. AUC i czas przetwarzania [5].

Artykuł wybrano jako pierwotne źródło jednego z dwóch głównych algorytmów projektu. Uzasadnia zainteresowanie metodą, która nie wymaga etykiety każdej próbki ani obliczania wszystkich wzajemnych odległości. Podpróbkowanie i zespół drzew są istotne dla analizy regularnie napływających pomiarów. Przedstawione wyniki dotyczą jednak zbiorów autorów i nie dowodzą przewagi na danych z trzech węzłów ESP32.

W projekcie metoda będzie oceniać wspólny wektor sześciu kanałów, co pozwala badać nietypowe kombinacje wartości, a nie wyłącznie przekroczenia progów pojedynczych czujników. Należy zapisywać liczbę drzew, wielkość podpróbki i ziarno losowania, aby umożliwić odtworzenie porównania z One-Class SVM. Przy przekazywaniu wyniku do HMI trzeba uwzględnić konwencję implementacji: oryginalny wynik anomalii i wynik scikit-learn nie mają tej samej orientacji. Sam detektor nie identyfikuje przyczyny zdarzenia i bez cech czasowych nie modeluje kolejności próbek; te ograniczenia powinny towarzyszyć interpretacji alarmów.

1.2.2 Arnau Muñoz i współautorzy: jakość danych w infrastrukturze IoT

Artykuł *Anomaly detection system for data quality assurance in IoT infrastructures based on machine learning* opisuje system wykrywający podejrzane dane w heterogenicznej infrastrukturze Uniwersytetu w Alicante. Autorzy łączą zbieranie wiadomości przez MQTT, analizę i przygotowanie cech z detekcją wykorzystującą Isolation Forest. Celem jest ochrona jakości danych przekazywanych aplikacjom; wykrycie nietypowego pakietu nie jest utożsamiane ze szczegółową diagnozą awarii [6].

Publikację wybrano jako najbliższy architektonicznie przykład zastosowania. Pokazuje ona, że użyteczny detektor wymaga również warstwy akwizycji, przechowywania i przygotowania danych. Autorzy analizują dane pochodzące z wielu systemów kampusowych, dlatego ich liczby cech, skala infrastruktury i wyniki nie mogą być bezpośrednio przenoszone na trzy węzły ESP32.

Konkretnym wkładem jest wzorzec oddzielenia odbioru telemetrii od analizy ML. W naszym systemie jeden wiersz CSV opisuje pojedynczy kanał, więc przed detekcją konieczne jest utworzenie spójnego wektora pomiarowego. Informacje o urządzeniu, kanale i czasie odbioru należy zachować do późniejszej interpretacji alarmu. W odróżnieniu od celu filtrowania strumienia przedstawionego przez autorów proponuje się pozostawienie surowych danych i oznaczanie podejrzanych obserwacji w HMI. Umożliwi to sprawdzenie fałszywych alarmów i odróżnienie błędu danych od rzeczywistej zmiany warunków fizycznych.

1.2.3 Schölkopf i współautorzy: One-Class SVM

Praca *Estimating the Support of a High-Dimensional Distribution* dotyczy estymacji obszaru przestrzeni, w którym skupia się rozkład danych. Autorzy konstruują funkcję o różnych znakach wewnątrz i poza wyznaczonym obszarem, wykorzystując jądra oraz optymalizację kwadratową. Jest to rozszerzenie idei maszyn wektorów nośnych na dane bez etykiet klas. Publikacja zawiera opis algorytmu, analizę teoretyczną i doświadczenia na danych sztucznych oraz rzeczywistych [7].

Źródło wybrano, ponieważ wyjaśnia podstawy drugiego badanego detektora, a nie jedynie sposób wywołania biblioteki. One-Class SVM opisuje obszar zachowania odniesienia, co stanowi wartościowy kontrast wobec mechanizmu izolacji obserwacji. Parametr ν wiąże się z górnym ograniczeniem udziału błędów treningowych i dolnym ograniczeniem udziału wektorów nośnych, przy założeniach podanych przez autorów. Nie jest gwarantowanym odsetkiem rzeczywistych awarii ani fałszywych alarmów w przyszłych danych.

Dla projektu wkład obejmuje uzasadnienie uczenia na możliwie wiarygodnym fragmencie normalnej pracy i kontroli zanieczyszczenia treningu. W wariancie z jądrem RBF trzeba dobrać ν oraz parametr γ, a także zadbać o skalowanie kanałów o różnych jednostkach. Wyniki należy porównać z Isolation Forest na identycznych przedziałach czasu, mierząc również koszt inferencji na serwerze. HMI powinien prezentować wynik jako ocenę zgodności z wyuczonym obszarem, bez nadawania mu znaczenia skalibrowanego prawdopodobieństwa awarii urządzenia.

1.2.4 Schmidl, Wenig i Papenbrock: porównanie detekcji czasowej

Praca *Anomaly Detection in Time Series: A Comprehensive Evaluation* porównuje 71 algorytmów na 976 zbiorach szeregów czasowych. Autorzy porządkują rodziny metod i analizują skuteczność, czas działania oraz zależność wyników od właściwości danych. Rozróżniają m.in. anomalie punktowe, nietypowe fragmenty przebiegów i anomalie związane z zależnościami między kanałami. Udostępnione materiały uzupełniające pozwalają prześledzić organizację porównania i wykorzystane implementacje [8].

Artykuł wybrano, ponieważ przeciwdziała ocenianiu algorytmu wyłącznie na podstawie jednego udanego przykładu. Wynik konkretnej implementacji zależy także od parametrów, reprezentacji danych i rodzaju zakłócenia. Rozległy benchmark stanowi wzorzec rzetelnego porównania, ale jego ranking nie rozstrzyga automatycznie, który model sprawdzi się w lokalnej sieci ESP32. Projekt ma mniejszą skalę i własny zestaw kanałów pomiarowych.

Konkretny wkład to plan oceny Isolation Forest i One-Class SVM w kilku scenariuszach, np. krótkiego wstrząsu, zmiany oświetlenia i utrzymującego się odchylenia. Są to proponowane scenariusze projektu, a nie deklaracja wykonania benchmarku autorów. Przy dostępnych etykietach należy raportować precyzję, czułość i F1 oraz jasno określić, czy ocena dotyczy próbek, czy całych zdarzeń. Bez etykiet nie należy przedstawiać udziału alarmów jako skuteczności. Dla użytkownika HMI istotne będą dodatkowo liczba fałszywych powiadomień i opóźnienie wykrycia, zaś dla serwera koszt przetwarzania nowych pomiarów.

1.3 Dokumentacje i rozwiązania techniczne

Dokumentacje uzupełniają podstawy naukowe o informacje potrzebne do budowy systemu. Obejmują implementację modeli, akwizycję i przesyłanie pomiarów oraz rozwiązania stanowiące punkt odniesienia dla interfejsu HMI.

1.3.1 scikit-learn: Novelty and Outlier Detection

Dokumentacja rozróżnia wykrywanie obserwacji odstających w potencjalnie zanieczyszczonym zbiorze oraz wykrywanie nowych nietypowych obserwacji po uczeniu na danych odniesienia. Omawia m.in. Isolation Forest i One-Class SVM oraz wspólny interfejs estymatorów. W predykcji etykieta $+1$ oznacza obserwację uznaną za normalną, a -1 za odstającą; funkcja decyzyjna pozwala pracować również z wynikiem ciągłym [9].

Źródło wybrano jako bezpośrednią podstawę implementacji i interpretacji wyników. Artykuły naukowe definiują metody, natomiast dokumentacja wyjaśnia zachowanie udostępnionego oprogramowania. Jest to istotne zwłaszcza dla Isolation Forest: parametr `contamination` wpływa na próg decyzji, a nie dostarcza etykiet rzeczywistych zdarzeń. Dokumentacja wskazuje również wrażliwość One-Class SVM na obserwacje odstające i potrzebę dostosowania parametrów do zadania.

W projekcie źródło pozwala ujednolicić warstwę komunikacji modeli z serwerem i HMI. Należy zapisywać zarówno wynik ciągły, jak i decyzję, użyty próg oraz identyfikator modelu. Ujemny wynik funkcji decyzyjnej powinien być interpretowany zgodnie z API, aby nie odwrócić znaczenia alarmów. Wyników obu modeli nie należy traktować jako porównywalnych prawdopodobieństw. W raporcie eksperymentalnym trzeba też podać rzeczywiście użytą wersję biblioteki; odwołanie do dokumentacji `stable` samo w sobie nie ustala wersji środowiska. Pozwoli to odtworzyć predykcje i sprawdzić zgodność wyników prezentowanych na wykresach.

1.3.2 Eclipse Mosquitto: broker MQTT

Mosquitto jest otwartym brokerem implementującym MQTT, czyli komunikację w modelu publikowania i subskrypcji. Nadawcy publikują wiadomości na tematach, a broker przekazuje je zainteresowanym odbiorcom. Projekt udostępnia również bibliotekę klienta i narzędzia `mosquitto_pub` oraz `mosquitto_sub`. Dokumentacja i repozytorium opisują komponent komunikacyjny, a nie gotowy system analizy pomiarów lub detekcji anomalii [10].

Źródło wybrano, ponieważ broker stanowi naturalny element architektury z kilkoma ESP32 i centralnym odbiorcą. Rozdzielenie publikujących urządzeń od subskrybującego serwera ułatwia dodawanie kanałów oraz niezależne testowanie odbioru telemetrii. Narzędzia konsolowe umożliwiają sprawdzenie tematów i treści wiadomości przed uruchomieniem analizy ML. Użyteczność Mosquitto nie oznacza, że sam broker zapewnia poprawność fizyczną danych lub synchronizację pomiarów z różnych urządzeń.

Wkład w projekt to uzasadnienie warstwy transportowej i sposobu testowania całego przepływu. Odbiorca powinien zachować temat MQTT, identyfikator urządzenia, numer sekwencyjny i czas odbioru, aby umożliwić analizę przerw lub powtórzeń. Dobór parametrów dostarczania i ponownego łączenia należy udokumentować jako konfigurację projektu, bez zakładania, że przybycie wiadomości oznacza jednoczesne próbkowanie. W HMI warto oddzielić brak aktualnych danych od alarmu modelu: utrata połączenia ESP32

nie musi generować nietypowego wektora, lecz może uniemożliwić jego utworzenie. Broker i detektor rozwiązują zatem różne, uzupełniające się zadania.

1.3.3 ThingsBoard: platforma IoT jako punkt odniesienia

Dokumentacja ThingsBoard opisuje platformę obejmującą zarządzanie urządzeniami, zbieranie telemetrii, pulpity wizualizacyjne oraz obsługę alarmów i reguł przetwarzania. Materiały użytkownika pokazują, jak powiązać dane z urządzeniami i prezentować ich stan w aplikacji. Jest to przykład szerszego rozwiązania IoT, którego funkcje wykraczają poza pojedynczy model detekcji [11].

Platformę wybrano jako punkt odniesienia dla zakresu funkcjonalnego HMI. Porównanie z istniejącym produktem pomaga określić, co powinno być dostępne dla operatora poza samym wykresem. Dokumentacja funkcji nie stanowi jednak niezależnego pomiaru jakości platformy ani dowodu skuteczności Isolation Forest lub One-Class SVM. Przy porównywaniu rozwiązań trzeba wskazać konkretną edycję i konfigurację, ponieważ nie wszystkie możliwości całego ekosystemu należą do jednego wariantu produktu.

Konkretnym wkładem w projekt jest zestaw kryteriów projektowania interfejsu: identyfikacja węzła, aktualna wartość z jednostką, historia pomiarów, czas wystąpienia alarmu i możliwość obejrzenia jego kontekstu. Proponowane HMI powinno ponadto prezentować model i próg, które doprowadziły do zgłoszenia, aby wspierać porównanie detektorów. Własny system można ocenić względem ThingsBoard pod kątem tych funkcji, bez twierdzenia, że przewyższa platformę komercyjną. Uzasadnieniem jego budowy jest kontrolowany eksperyment z własnym przygotowaniem danych i algorytmami, a nie konieczność odtworzenia wszystkich funkcji zarządzania rozbudowaną infrastrukturą IoT.

1.3.4 Grafana: wizualizacja i organizacja alarmów

Dokumentacja Grafana Alerting opisuje reguły, które okresowo odpytują źródła danych i sprawdzają warunki. Rozróżnia regułę alarmową, konkretne instancje alarmu oraz kierowanie powiadomień do punktów kontaktowych. Powiadomienia można organizować bezpośrednio lub za pomocą polityk. Pozwala to oddzielić obliczanie warunku od sposobu informowania użytkownika [12].

Źródło wybrano jako praktyczny wzorzec warstwy alarmowania i punkt odniesienia dla HMI. Samo wykrycie odchylenia przez model nie określa jeszcze, kiedy wysłać wiadomość i jak uniknąć wielu zgłoszeń dotyczących tego samego zdarzenia. Dokumentacja wyjaśnia te pojęcia na poziomie systemowym, ale nie jest publikacją potwierdzającą skuteczność algorytmów ML. Przedstawione mechanizmy nie oznaczają również, że Grafana jest już elementem implementacji projektu.

W projekcie proponuje się wykorzystać ten podział do zaprojektowania osobnego rejestru wyników i rejestru zdarzeń. Kolejne alarmowe próbki z jednego okresu zakłócenia mogą być prezentowane operatorowi jako jedno zdarzenie z czasem rozpoczęcia i zakończenia, zamiast wielu niezależnych wiadomości. Jest to propozycja dla własnego systemu, wymagająca określenia reguł łączenia zdarzeń. Na wspólnej osi czasu HMI powinien pokazywać pomiary, wynik detektora i próg. Przerwa w telemetrii wymaga osobnej obsługi, ponieważ brak danych nie jest równoznaczny z normalną pracą. Taka organizacja ułatwi ocenę użyteczności alarmów oraz powiązanie ich z eksperymentami fizycznymi.

1.3.5 Espressif: biblioteki Arduino-ESP32

Oficjalna dokumentacja Arduino-ESP32 opisuje biblioteki udostępniające funkcje układów ESP32 przez interfejsy Arduino. Zestawienie obejmuje m.in. GPIO, I2C, SPI, timery i komunikację Wi-Fi oraz pokazuje różnice w dostępności funkcji między układami rodziny. Materiały producenta są podstawą sprawdzania możliwości wybranego mikrokontrolera i korzystania z jego interfejsów [13].

Źródło wybrano ze względu na zgodność z warstwą sprzętową projektu. Trzy węzły muszą regularnie odczytywać czujniki i przekazywać pomiary do serwera, dlatego sposób obsługi magistral i połączenia sieciowego wpływa na jakość danych wejściowych ML. Dokumentacja rdzenia Arduino nie zastępuje jednak dokumentacji konkretnych czujników AHT20, BMP280, BH1750 i ADXL345 ani dokumentacji wybranej biblioteki klienta MQTT. Nie należy przypisywać jej gotowych sterowników lub algorytmów, których nie opisuje.

Konkretnym wkładem jest możliwość sprawdzenia interfejsów i ograniczeń sprzętowych przed interpretacją nietypowych odczytów. W oprogramowaniu węzłów należy jednoznacznie określić jednostki, częstotliwość raportowania i znaczenie znaczników czasu, a dla akcelerometru również sposób wyznaczania RMS i maksimum. Są to wymagania projektowe wynikające z potrzeby porównywalności pomiarów. Numer sekwencyjny i czas urządzenia pomogą rozpoznać restart lub lukę w transmisji; czas odbioru na serwerze pozostaje osobną informacją. Tak przygotowana telemetria ułatwi oddzielenie problemów akwizycji od alarmów Isolation Forest i One-Class SVM oraz prezentację wiarygodnego stanu węzłów w HMI.

1.4 Wnioski dla projektu

Literatura uzasadnia porównanie dwóch mechanizmów detekcji na wspólnej reprezentacji danych. W metodologii należy rozdzielić przygotowanie pomiarów, uczenie, wybór progu i obsługę alarmów. Skuteczność trzeba ocenić na danych projektu; nie wynika ona automatycznie z cytowania publikacji.

2 Metodologia zbierania i podziału danych

Pomiary z trzech węzłów ESP32 zapisano na serwerze i zestawiono ze sobą na podstawie czasu odbioru. Dane podzielono chronologicznie na trzy zbiory w proporcji 70/15/15: najwcześniejsze pomiary przeznaczono na trening, kolejne na walidację, a najnowsze na test.

2.1 Zbieranie danych

W systemie działają trzy węzły ESP32: pierwszy z czujnikami AHT20 i BMP280, drugi z czujnikiem oświetlenia BH1750, a trzeci z akcelerometrem ADXL345. Telemetria trafia przez MQTT na serwer i jest zapisywana w pliku CSV. W repozytorium plik `data/raw/telemetry.csv` zawiera czas odbioru `received_at`, temat MQTT, identyfikator urządzenia, nazwę kanału, czas urządzenia `ts_ms`, numer sekwencyjny, wartość oraz jednostkę. Każdy wiersz opisuje jeden kanał, a nie pełny wektor wszystkich pomiarów.

Zebrany zbiór obejmuje około 39 godzin pomiarów.

2.2 Synchronizacja i reprezentacja pomiarów

Dane z poszczególnych kanałów sprowadzono do wspólnej siatki czasowej o kroku 5 s. Jako punkt odniesienia przyjęto czas odbioru na serwerze, dzięki czemu możliwe było zestawienie kanałów z różnych urządzeń. Nie jest to jednak równoznaczne z jednoczesnym próbkowaniem: opóźnienia w transmisji i różne częstotliwości raportowania nadal wprowadzają pewną niepewność.

Każda przygotowana próbka składa się z sześciu cech:

1. `AHT20_temperature` – temperatura;
2. `AHT20_humidity` – wilgotność względna;
3. `BMP280_pressure` – ciśnienie atmosferyczne;
4. `BH1750` – natężenie oświetlenia;
5. `ADXL345_accel_rms` – raportowana miara RMS przyspieszenia;
6. `ADXL345_accel_max` – raportowana maksymalna wartość przyspieszenia.

Znacznik `received_at` służy wyłącznie do uporządkowania danych i nie jest cechą modelu. Obie miary z akcelerometru potraktowano jako osobne kanały.

2.3 Podział chronologiczny i oczyszczanie

Obserwacje podzielono chronologicznie w proporcji 70/15/15: najwcześniejsza część jest przeznaczona do uczenia, następna do walidacji, a ostatnia do końcowego testu. Nie zastosowano losowego mieszania próbek. Sąsiednie odczyty są ze sobą powiązane w czasie, więc przy losowym podziale bardzo podobne fragmenty przebiegu mogłyby trafić jednocześnie do zbioru treningowego i do oceny. Podział chronologiczny lepiej odpowiada sytuacji, w której model analizuje pomiary z późniejszego okresu, choć nie eliminuje całkowicie zależności między zbiorami.

Zbiór walidacyjny będzie służył do obserwowania zachowania modelu i doboru parametrów. Zbiór testowy pozostaje zarezerwowany do oceny końcowej, po podjęciu tych decyzji, i nie powinien służyć do dobierania progu. Podane proporcje dotyczą samego podziału chronologicznego; po przygotowaniu i oczyszczeniu danych liczebności zbiorów mogą się od nich nieznacznie różnić.

Potencjalne wstrząsy usuwa się wyłącznie ze zbioru treningowego, aby model nie uczył się zakłóceń jako zachowania referencyjnego. Ze zbiorów walidacyjnego i testowego nie usuwa się obserwacji tylko dlatego, że wyglądają nietypowo, ponieważ sztucznie ułatwiłoby to ocenę. Regułę oczyszczania należy ustalić na podstawie zbioru treningowego i wiedzy o eksperymencie, bez korzystania z późniejszych wyników. Nie stanowi ona jednak potwierdzenia etykiet anomalii.

Liczebności zbiorów po przygotowaniu danych zestawiono w tabeli. Dotyczą one gotowych wektorów sześciu cech, a nie surowych wierszy telemetrii.

Zbiór	Liczba próbek	Liczba cech
Treningowy	19 634	6
Walidacyjny	4 222	6
Testowy	4 222	6

Literatura

[1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020. Wydanie amerykańskie. Materiały autorów: `https://aima.eecs.berkeley.edu/`.

[2] Zhi-Hua Zhou. *Machine Learning*. Springer Singapore, 2021. `https://doi.org/10.1007/978-981-15-1967-3`.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `https://www.deeplearningbook.org/`.

[4] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, 3 edition, 2022. Materiały autora do wydania trzeciego: `https://github.com/ageron/handson-ml3`.

[5] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE, 2008. `https://doi.org/10.1109/ICDM.2008.17`.

[6] Lucia Arnau Muñoz, José Vicente Berná Martínez, Francisco Maciá Pérez, and Iren Lorenzo Fonseca. Anomaly detection system for data quality assurance in IoT infrastructures based on machine learning. *Internet of Things*, 25:101095, 2024. Numer artykułu: 101095. `https://doi.org/10.1016/j.iot.2024.101095`.

[7] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001. `https://doi.org/10.1162/089976601750264965`.

[8] Sebastian Schmidl, Phillip Wenig, and Thorsten Papenbrock. Anomaly detection in time series: A comprehensive evaluation. *Proceedings of the VLDB Endowment*, 15(9):1779–1797, 2022. `https://doi.org/10.14778/3538598.3538602`.

[9] scikit-learn developers. Novelty and outlier detection. Dokumentacja scikit-learn, kanał stable. `https://scikit-learn.org/stable/modules/outlier_detection.html`. Dostęp: 11.11.2026.

[10] Eclipse Foundation. Eclipse Mosquitto: broker MQTT i repozytorium projektu. Oficjalna strona i kod źródłowy. `https://mosquitto.org/`; `https://github.com/eclipse-mosquitto/mosquitto`. Dostęp: 11.11.2026.

[11] ThingsBoard Authors. ThingsBoard: User guide. Oficjalna dokumentacja platformy. `https://thingsboard.io/docs/user-guide/`. Dostęp: 11.11.2026.

[12] Grafana Labs. Introduction to Grafana Alerting. Oficjalna dokumentacja, kanał latest. `https://grafana.com/docs/grafana/latest/alerting/fundamentals/`. Dostęp: 11.11.2026.

[13] Espressif Systems. Arduino-ESP32: Libraries. Oficjalna dokumentacja, kanał latest. `https://docs.espressif.com/projects/arduino-esp32/en/latest/libraries.html`. Dostęp: 11.11.2026.